one bit went wrong

digitalisierung und programmierung · prof. dr. nicolas meseth

1. it will go wrong
2. how you notice
3. what it costs
4. notice or repair

part 1

it will go wrong

torn and scratched.
readable anyway.
why?



image: ai-generated (gpt-image-2)

try it: a piece of this code is missing



squares in the code	1 089
squares torn out	172
bytes of data	36
bytes of protection	64

and it still reads

`docs.lifi-project.de`

whoever designed this assumed it would be damaged, and paid for it in advance.

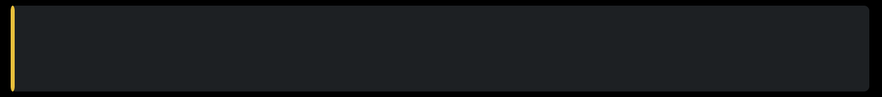
qr version 4, level h. checked with a decoder: blurred, rotated, shrunk to 260 px, saved as jpeg quality 45

errors are not bad luck, they are arithmetic

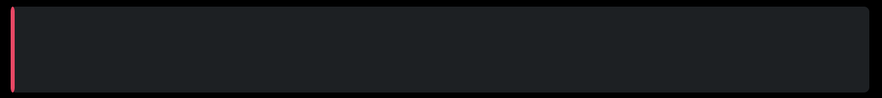
a link that gets 999 bits out of 1000 right



10 characters
80 bits — 92.3 %



the same, 100 times
8 000 bits — 0.03 %



the payload of the final
16 384 bits — 0.0000076 %

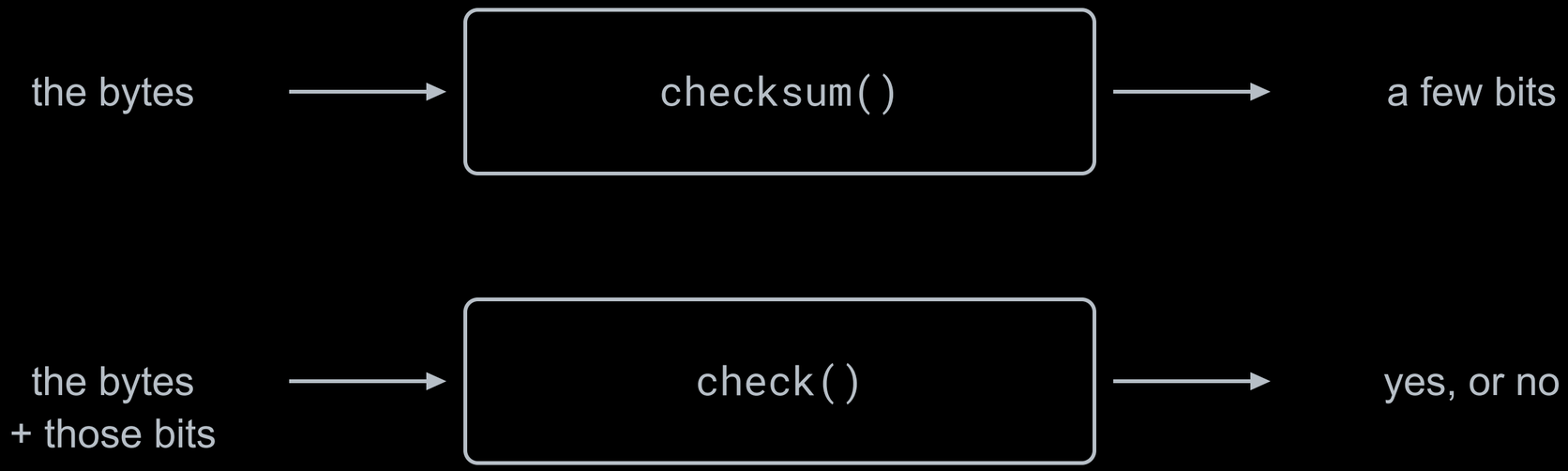
the chance that everything arrives untouched. nothing is broken here: this is multiplication.

the question is not how to avoid errors. it is how to notice them.

at an error rate of 0.1 % per bit

it will go wrong

the check as a box, twice



the second box never repairs anything. it answers one question.

redundancy is bits that carry nothing new, and earn their place by what they rule out.

part 2

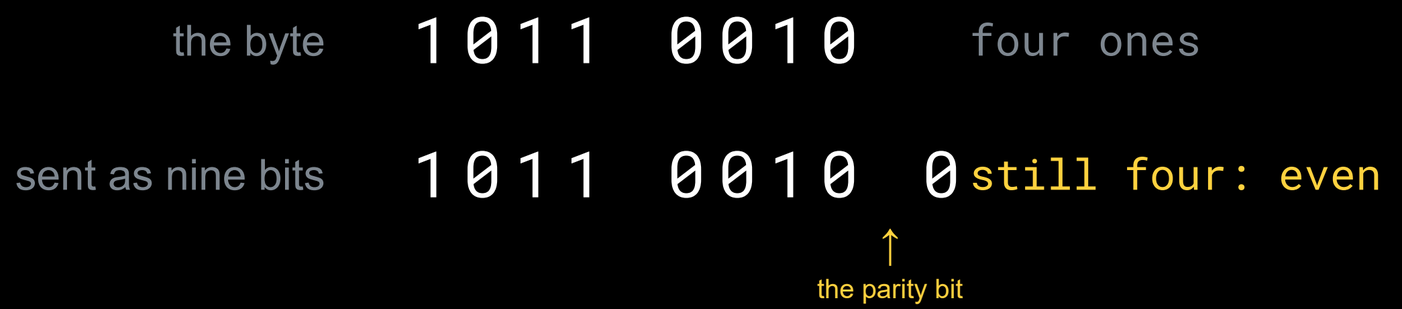
how you notice

one bit that only says: even or odd

the byte `1011 0010` four ones

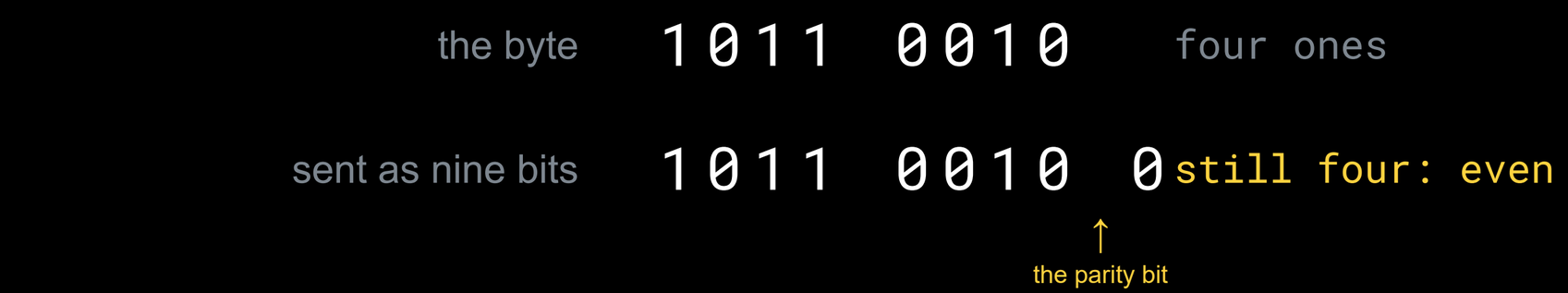
one extra bit per byte, and every single flipped bit shows.

one bit that only says: even or odd



one extra bit per byte, and every single flipped bit shows.

one bit that only says: even or odd



the receiver counts the ones, finds an odd number, and knows: this byte is broken.

one extra bit per byte, and every single flipped bit shows.

sent	1011 0010 0	four ones: even
two bits flip	1010 0110 0	four ones: even

the count is right again. the check says nothing, and the byte is wrong.

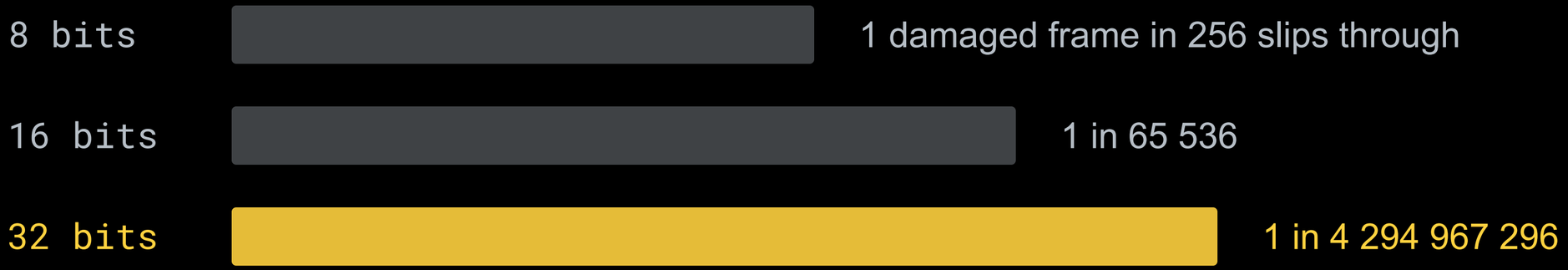
parity catches every odd number of flips and misses every even one. that is what one extra bit buys.

a check that misses half the cases is not a guarantee. it is a filter.

from one bit to a checksum



computed from every byte of the frame, by a rule both sides agreed on



crc-32, the 32-bit version, is what png and zip carry. more check bits do not catch more errors: they let fewer slip past.

more check bits do not catch more errors. they let fewer slip past.

a fingerprint for a whole file

the letter a 1 byte	→	ca978112ca1bbdcafac231b39a23dc4d a786eff8147c4e72b9807785afee48bb
meet at noon 12 bytes	→	50a8d1d0939b05a7fb60b34d4fb18805 a1ced867736de235c7095a03bd7d1c25
the parrot, raw 196 662 bytes	→	c96f9a5adeeda470bd66eb13c0a78607 097890ae957b6f911407e73d6055f94f

always 64 characters. always.

a hash is a fingerprint of the bytes: a fixed rule squeezes any amount of data into one short number.

any amount of data in. always the same short number out.

sha-256, the hash your computer uses for downloads · computed 15 sept 2026

and one bit changes everything

the parrot	`c96f9a5adeeda470bd66eb13c0a78607` `097890ae957b6f911407e73d6055f94f`
one bit flipped	`c793da57f7f8bf7fc090e0ce574684f5` `b196aee88e9130c25b6da140e797c13f`

7 of 64 characters still match, and that is pure chance: with 16 possible characters, four would match between any two files.

one bit in 1 573 296, and nothing of the fingerprint survives.

no similarity survives. that is exactly what makes it a test.

grey: characters that happen to match · yellow: characters that differ

three checks, three jobs

the check	covers	and tells you
parity bit	`one byte`	an odd number of bits flipped
checksum	`one frame`	this frame is broken: send it again
hash	`the whole file`	these 2 kb are exactly those 2 kb

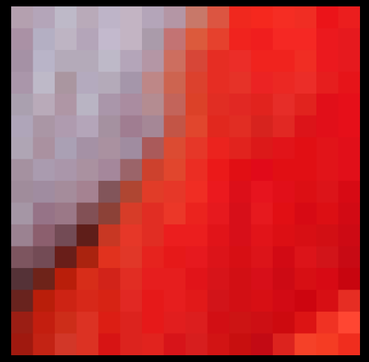
in the final the hash decides. the frame checksums are what get you there without sending everything twice.

the hash decides. the frame checksums are how you get there.

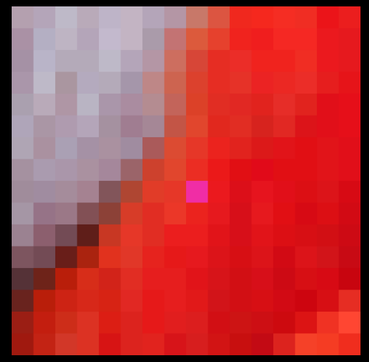
one flipped bit in the raw picture



the file after one flipped bit



as sent



as received

zoomed to 16 × 16 points

`(240, 45, 38) → (240, 45, 166)`

one colour value of 196 608 is wrong. the file opens. nobody sees it.

a picture that displays proves nothing.

real file, 256 × 256 points · parrot: ai-generated (gpt-image-2)

how you notice

the same bit, in the packed file



raw: opens, one wrong point

unrecognized data stream
contents when reading image file

png: does not open at all

```
crc-32 stored a831c3c1
crc-32 computed cb5ebce6
```

packed data has no air left: every byte hangs on the ones before it. png notices by itself and refuses.

packed data has no air left. one wrong bit, and the rest is gone.

measured 15 sept 2026 · same picture, same kind of error, different format

how you notice

try it: break a byte yourself

picture:   

pixels: 8 × 8 | 16 × 16 | 32 × 32

what a viewer makes of the bytes



click a pixel

open as text | reset file

the bytes (a hex editor)

```
0000   42 4d 36 03 00 00 00 00 00 00 36 00 00 00 28 00
0010   00 00 10 00 00 00 10 00 00 00 01 00 18 00 00 00
0020   00 00 00 03 00 00 c4 0e 00 00 c4 0e 00 00 00 00
0030   00 00 00 00 00 00 0a 0b 17 0e 11 1c 6b 69 62 45
0040   41 41 11 16 1e 12 19 20 13 19 20 10 18 1e 10 1a
0050   44 1b 22 42 0d 17 82 19 21 b0 1b 25 e4 26 3a f6
0060   28 3d f9 23 36 f9 0c 0d 1a 56 5e 66 a2 a4 a1 38
0070   3a 3e 10 17 1e 12 1c 21 1d 27 31 11 13 21 0b 0b
0080   1d 06 06 18 2e 33 62 28 30 91 1b 1e da 1e 26 e6
0090   26 3a f2 27 37 f5 21 25 32 9e a8 ae a2 a9 ab 55
00a0   57 55 16 17 1a 31 2e 31 3a 33 36 2a 24 27 25 20
00b0   23 1b 16 1f 40 3a 45 a2 94 a8 4b 4d c6 18 12 e0
00c0   17 0c df 12 0b da 50 57 62 9e ab b6 a1 a9 af 6a
00d0   69 68 12 10 11 4d 48 49 2d 27 26 2e 27 27 24 1e
00e0   20 25 1f 23 96 89 8e c5 b5 bf 97 90 c5 22 29 ee
00f0   1d 18 eb 1a 13 e9 83 8f 9c 9d ad bb a2 ad b5 90
0100   8e 88 42 3a 39 4d 45 43 2c 26 25 2c 25 25 2d 26
0110   26 75 6b 6e ca bb be ce bf c5 ca bd c7 72 7f de
0120   1f 28 f2 1e 24 f2 00 e7 b0 9e eb bd 98 e9 b7 9e
```

hover a byte

☐ file header ☐ info header ☐ pixel data ☐ selected pixel

822 bytes = 14 file header + 40 info header + 768 pixel data (16 × 16 pixels × 3 bytes)

photos: ai-generated (gpt-image-2)

part 3

what it costs

redundancy is paid in time

2 KiB over your link at 10 bit/s



noticing is nearly free. repairing without asking costs three times the link, every run, whether anything breaks or not.

noticing is nearly free. repairing by repetition is not.

try it: where did the bits go?

sender · 2 kb photo



one abstract channel symbol at a time

light link

waiting for the first symbol

receiver · checked payload only

0 / 16,384 bits accepted

packet 1 of 256 · attempt 1

preamble · 0 / 46 symbols

preamble	header	photo data	check
6 sym	4 sym	32 sym	4 sym

raw channel rate

10 bit/s

symbol rate × bits per symbol

effective photo rate

6.4 bit/s

after framing and expected retries

sent / useful

0 / 0

symbols sent / payload bits accepted

estimated time left

42:40

real link time, not display time

where the transmitted symbols went

0 committed symbols

accepted photo data 0 framing 0 failed attempts 0

pause start again 1× 100× 500×

photo: ai-generated (gpt-image-2)

part 4

notice or repair

repair without asking: say it three times

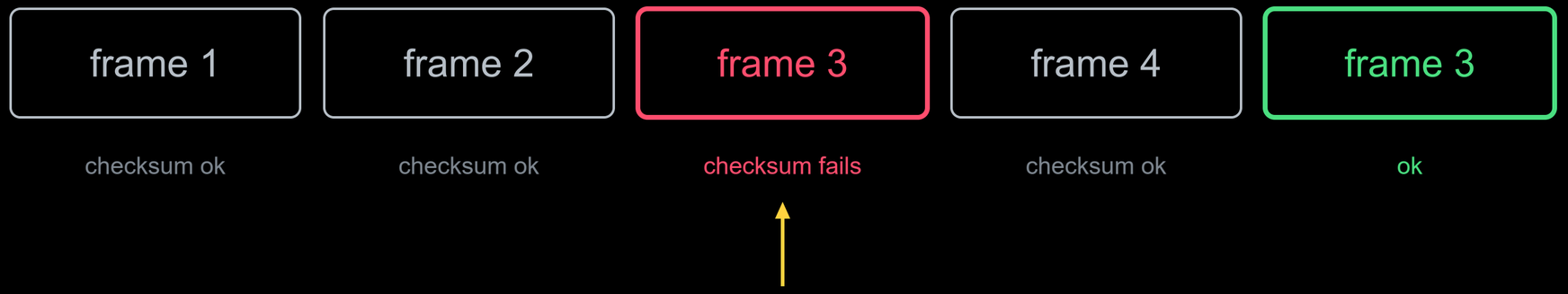


three times the bits · 82 minutes instead of 27 · and if two bits of the same triple flip, it repairs the wrong way

this is how a space probe does it: nobody can ask mars to repeat.

you can repair without a return channel. you pay for it on every bit, forever.

repair by asking: send that one again



costs 1.6 % while nothing breaks, and one frame when one breaks.

on a slow link, asking again beats paying in advance.

which one do you build?

a return channel: check, and ask again when it fails
no way to ask back: send everything more than once, and pay every time

you have a return channel. use it, keep the check small, and put the time you save into speed.

1 errors are arithmetic, not bad luck: enough bits, and one goes wrong
2 redundancy is bits that carry nothing new and rule things out
3 parity catches odd flips, a checksum catches almost everything, the hash decides
4 a picture that displays proves nothing
5 noticing costs 1.6 %. repeating everything costs 200 %. asking again costs only when it breaks.

this one you can see.
the dangerous one
you cannot.



image: ai-generated (gpt-image-2)